

Remarque

Ce document contient *une* solution pour chaque problème du test de novembre. Bien sûr, les algorithmes pour résoudre un problème ne sont pas uniques et votre algorithme, s'il est différent, est peut-être correct également.

Problème 1 : délimiteurs

```
def function(chaine, delim):
    index = 0
    temp = ""
    start = False
    while index < len(chaine):
        current = chaine[index]
        if current == delim:
            if start == True and len(temp) != 0:
                print temp
            start = True
            temp = ""
        else:
            temp = temp + current
        index = index + 1
```

Erreur fréquente. Il était clairement indiqué dans l'énoncé que la méthode devait afficher quelque chose. Or beaucoup d'étudiants n'ont pas employé de `print` mais seulement des `return`.

Problème 2 : ordonnancement de tâches

```
def creer_matrice(n_taches, contraintes):
    matrice = []
    # Création de la matrice vide
    for i in range(n_taches):
        line = []
        for j in range(n_taches):
            line.append(False)
        matrice.append(line)
    # Remplissage avec les contraintes
    for contrainte in contraintes:
        i, j = contrainte
        matrice[i][j] = True
    return matrice
```

```

def ordonnanceur(matrice):
    taille = len(matrice)
    taches_ok = [] # Contient l'ordre des tâches

    # Tant que toutes les tâches ne sont pas traitées
    while len(taches_ok) < taille:
        # Pour chaque tâche potentielle
        for i in range(taille):
            peut_faire = True
            # Recherche d'une tâche empêchant la tâche i d'être faite
            for j in range(taille):
                # S'il y a une contrainte et que la tâche n'est pas encore faite
                if matrice[i][j] == True and not (j in taches_ok):
                    peut_faire = False
                    break
            if peut_faire and not (i in taches_ok):
                taches_ok.append(i)
                break
    return taches_ok

```

Commentaires pour la fonction `creer_matrice`.

Essentiellement des effets de bord indésirables.

Commentaires pour la fonction `ordonnanceur`.

- face à une question où on doit décider (booléen) d'une propriété, et que dans un cas, on a la réponse pendant la boucle, et dans l'autre, à la fin, pensez à utiliser une variable booléenne *avant* la boucle pour initialiser. Par exemple, la fonction `all()` suivante met en avant ce qu'on attend :

```

def all(m):
    """ Affiche si tous les éléments de m sont vrais ou pas. """
    reponse = True
    for e in m:
        if not e:
            reponse = False
            break
    print reponse

```

Beaucoup ont tendance à faire un `if/else` et donc de "choisir" la réponse dès la première itération.

- Une erreur fréquente est d'oublier que les tâches ont pu être traitées avant : pour une tâche i , on parcourt la liste des tâches, et si une tâche j a une contrainte sur i , il ne faut pas oublier de regarder si j a déjà été réalisée ou pas (auquel cas, la contrainte tombe).
- L'erreur de "conception" la plus courante consistait à initialiser un ordre de réalisation des tâches avec `range()`, et puis de déplacer les différents "numéros" présents dans cet ordre en fonction des contraintes. Le problème, c'est qu'en cas de contraintes de la forme $(3, 4)$, $(2, 3)$, $(1, 2)$ et d'un ordre de base $[1, 2, 3, 4]$, cela peut mener à déplacer d'abord le 4 avant le 3 (donc $[1, 2, 4, 3]$), puis ils vont voir le 3 avant le 2 (donc $[1, 3, 2, 4]$), puis le 2 avant le 1 (donc $[2, 1, 3, 4]$) ce qui n'est pas du tout un bon ordre.

Problème 3 : nombres pairs

```
def somme_pair(t):
    if len(t) == 0:
        return 0
    else:
        s = 0
        if t[0] % 2 == 0:
            s = t[0]
        return s + somme_pair(t[1:])

def filtre_pair(t):
    if len(t) == 0:
        return []
    else:
        if t[0] % 2 == 0:
            return [t[0]] + filtre_pair(t[1:])
        else:
            return filtre_pair(t[1:])
```

Erreur fréquente. Le principal problème provient du non respect des consignes : aucun appel à une fonction ou une méthode des listes (à part `len`) n'était autorisé ; les fonctions devaient être strictement récursives (aucune boucle autorisée).

Problème 4 : le canonier

```
import math

def dpc(v, alpha):
    return (v*v*math.sin(2.*math.radians(alpha)))/9.81

def angle(d, a, b, v, r):
    found = False
    while not found:
        c = (a+b)/2.
        dpcc = dpc(v, c)
        if math.fabs(dpcc - d) < r:
            found = True
        elif d < dpcc:
            b = c
        else:
            a = c
    return c
```

Commentaires.

- La variable booléenne `found` indique si nous avons trouvé un angle permettant d'atteindre la cible. Elle est initialisée à `False`, car nous n'avons pas encore testé d'angle avant d'entrer dans la boucle ;

- Cette variable devient `True` lorsque l’on trouve un angle qui permet d’être suffisamment proche de la cible. C’est-à-dire lorsque la distance du point de chute du boulet avec l’angle `c` (`dpcc = dpc(v, c)`) est à une distance d’au plus `r` de `d` (`if math.fabs(dpcc - d) < r`);
- `c` est la moyenne des angles `a` et `b` (`(a+b)/2`);
- Si le boulet ne tombe pas suffisamment proche de la cible, nous regardons où le boulet est tombé par rapport à la cible. Si le tir était trop long (`elif d < dpcc :`), `c` devient la nouvelle valeur de (`b = c`). Notez que `b` représentait bien un angle pour lequel le boulet tombait trop loin. Enfin, `c` devient la nouvelle valeur de `a` (`a = c`) dans le cas contraire (`else :`).
- Lorsque l’on sort de la boucle, cela indique que la variable `found` est passée à `True` et donc que `c` est un angle permettant de toucher la cible. Il ne reste plus qu’à retourner `c`.

Erreurs fréquentes pour la fonction `angle`.

- La fonction était récursive et non itérative comme demandé.
- Une division entière était appliquée pour calculer la moyenne de `a` et `b`.
- Le rayon `r` donné en paramètre n’était pas utilisé. Une petite valeur arbitraire était utilisée.
- Un tableau contenant les valeurs entières de 0 à 45 (`range(0, 46)`) était utilisé pour assigner différentes valeurs à `a`, `b` ou `c`. La moyenne n’était donc pas calculée.
- `dpc` était recalculé dans la fonction `angle` alors qu’il suffisait d’appeler la fonction `dpc` précédente.
- Utilisation d’un paramètre `alpha` dans la fonction `angle`. Ce paramètre existe dans `dpc`, mais n’existe plus dans la fonction `angle`.

Erreurs fréquentes pour la fonction `dpc`.

- Oubli de transformer l’angle `alpha` en radians.
- Oubli de multiplier cet angle par 2 dans la fonction `sin`.
- Résultat imprimé (`print`) et non retourné (`return`).